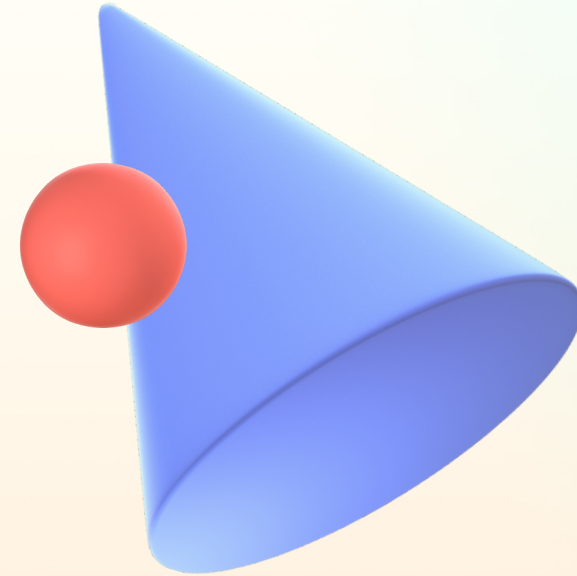
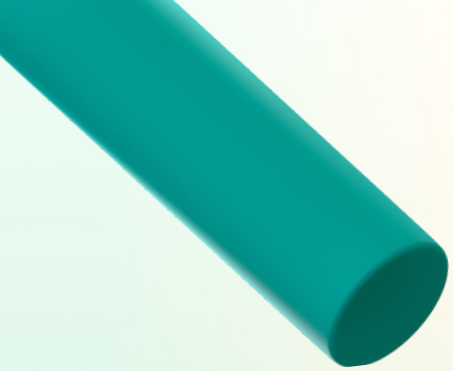
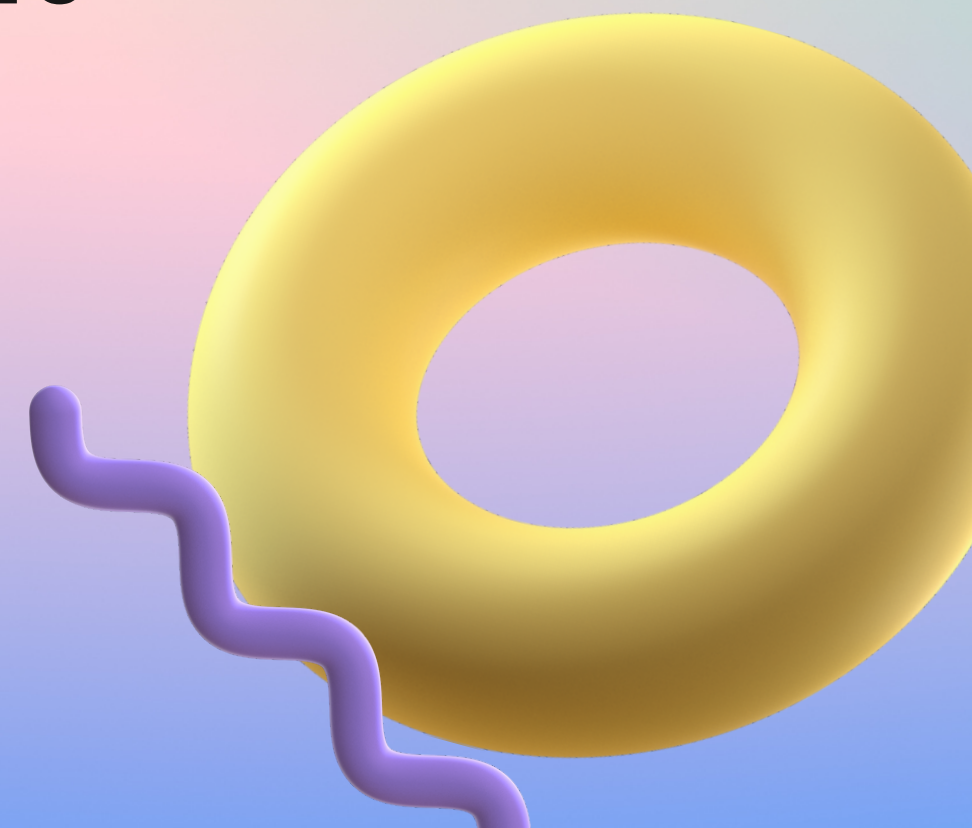
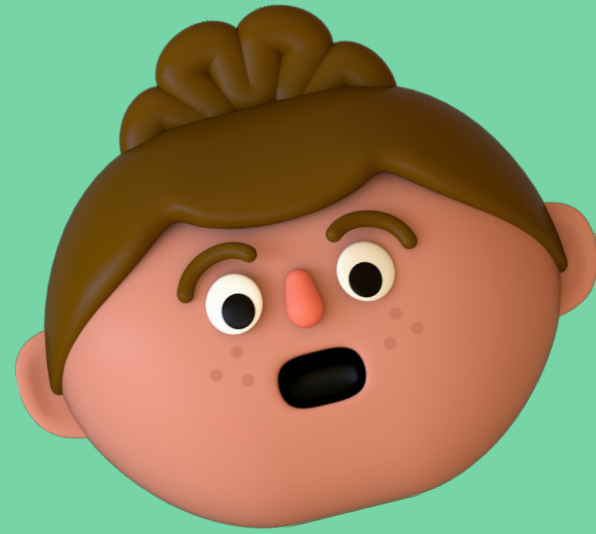


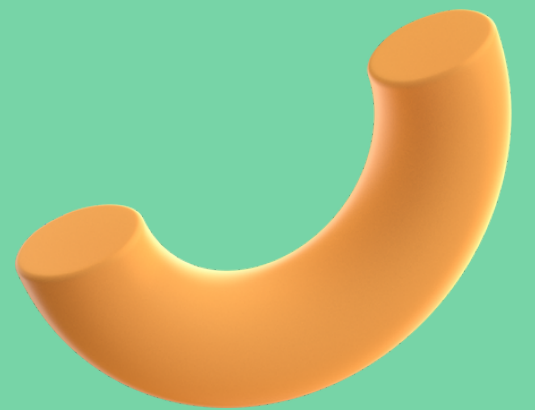
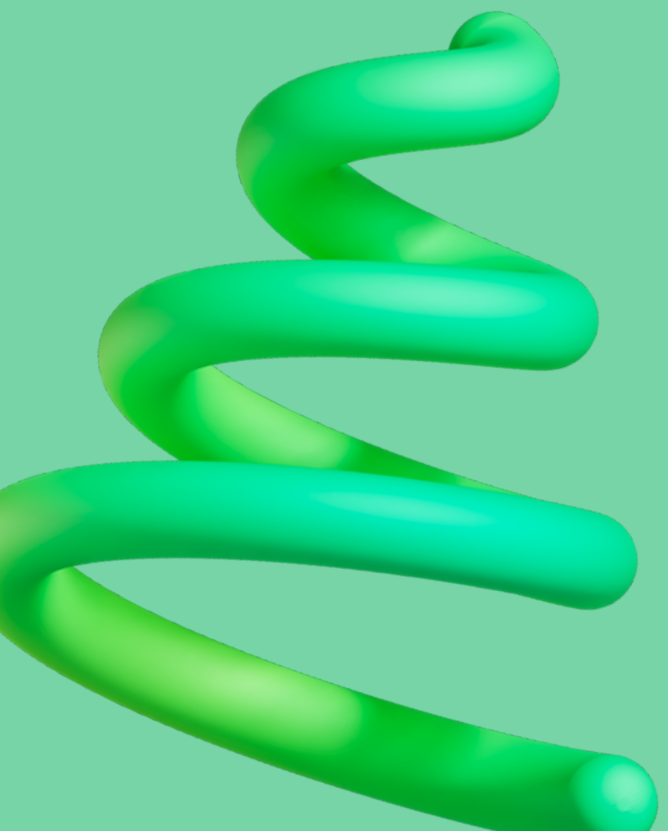
SUMMARIZER^{ble}

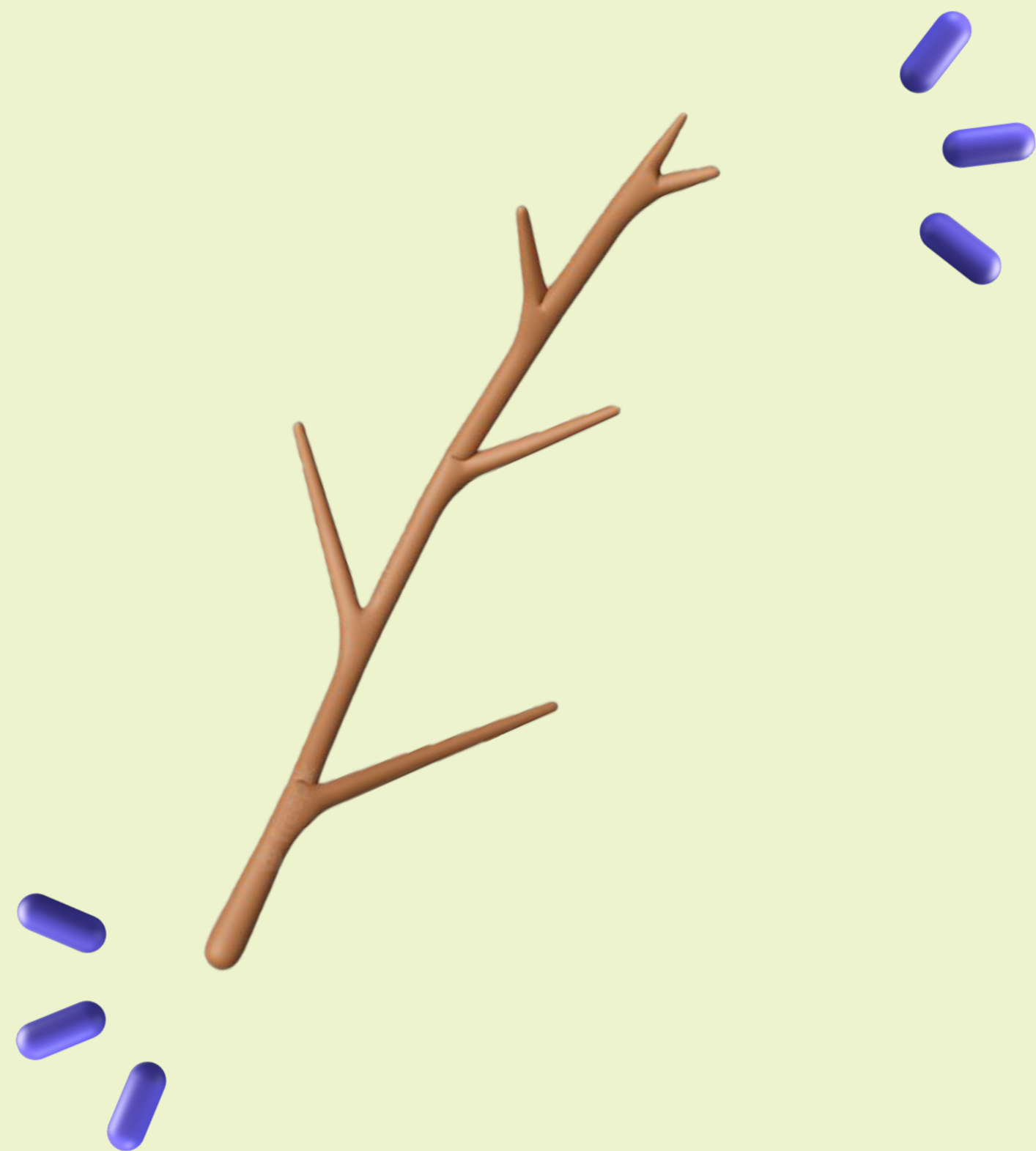
Sílvia Domingues | 13/02/23 | POWrangers

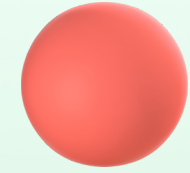




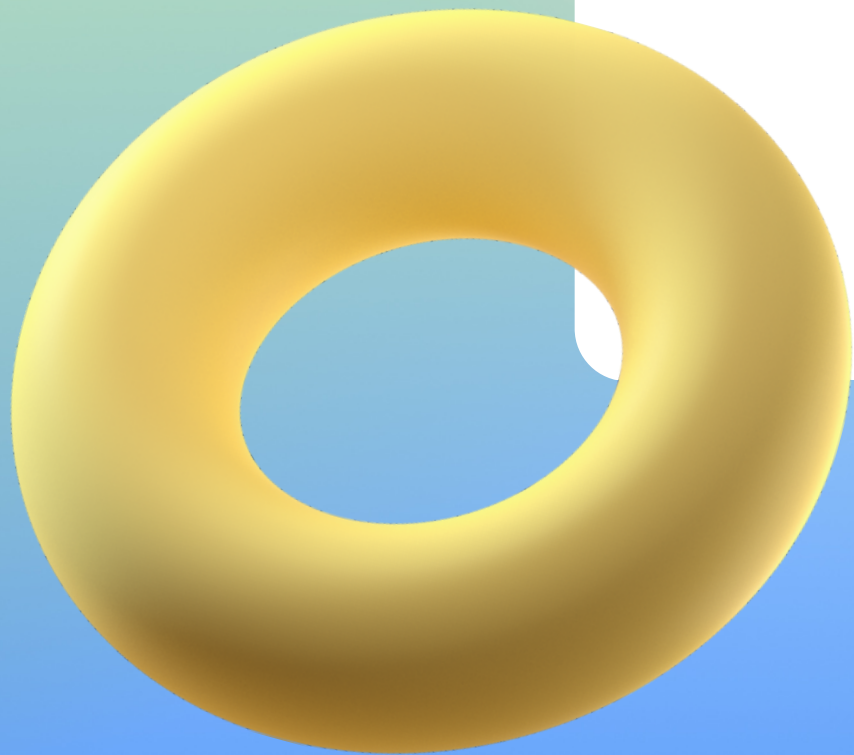
iterable





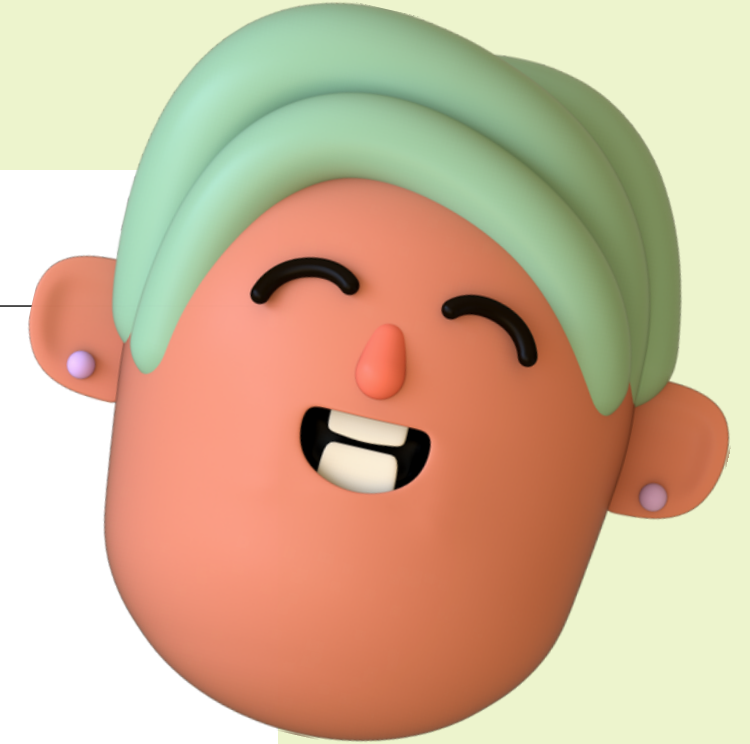


WHAT DOES THE ENHANCED LOOP DO?



The **enhanced loop** or **for-each loop** is a type of for loop that provides a convenient and concise way to iterate over the elements of an array, a collection, or any object that implements the Iterable interface in Java.



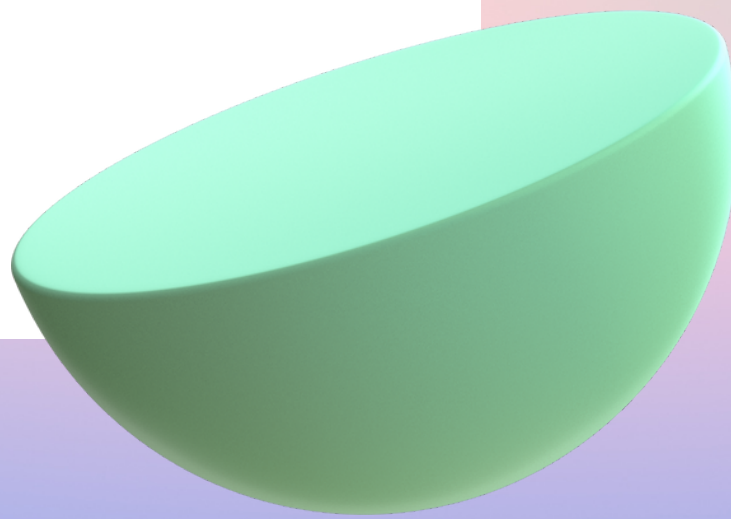
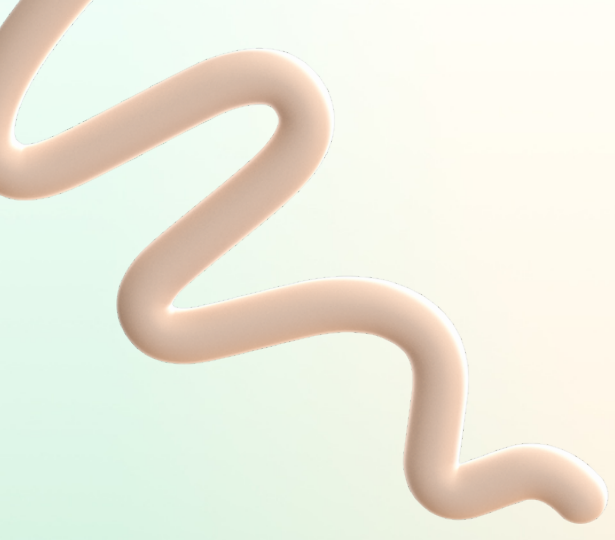


type of elements in the
collection

```
for(type item : array or collection) {  
}
```

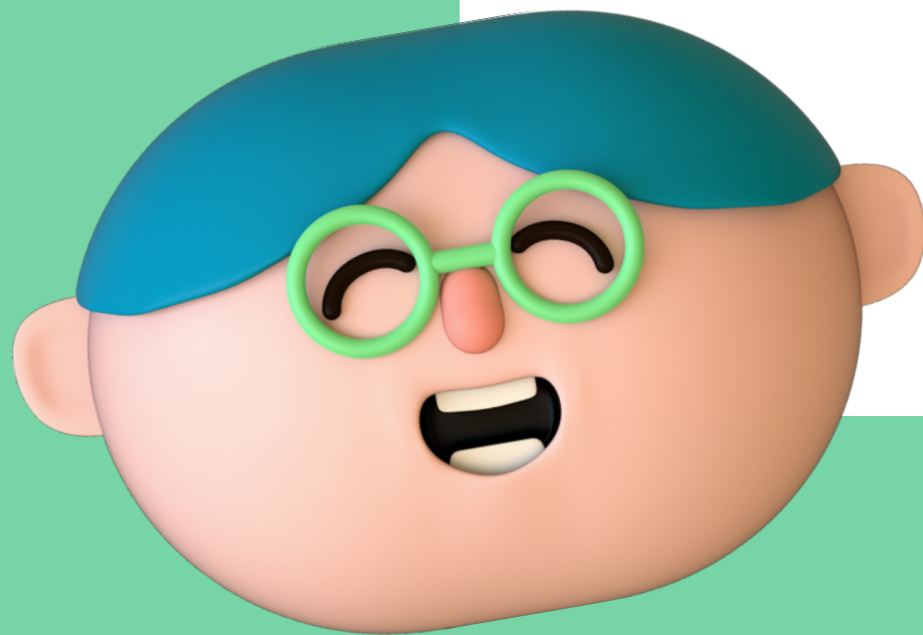
each item of array or
collection is assigned to
this variable as the loop
iterates through the elements

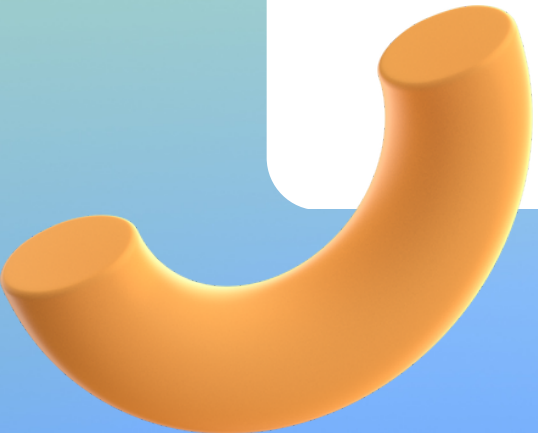
is the collection
or array over which
you want to iterate



WHAT IS AN ITERABLE INTERFACE?

The **Iterable interface** represents a container of elements that can be iterated over.
The Iterable interface provides an **iterator()** method which returns an **Iterator object**.





**AND WHAT ABOUT AN
ITERATOR INTERFACE?**

The **Iterator interface** is a core Java interface that provides a way to access elements of a collection one at a time. It's used to iterate over the elements of a collection, such as lists, sets, maps, etc.

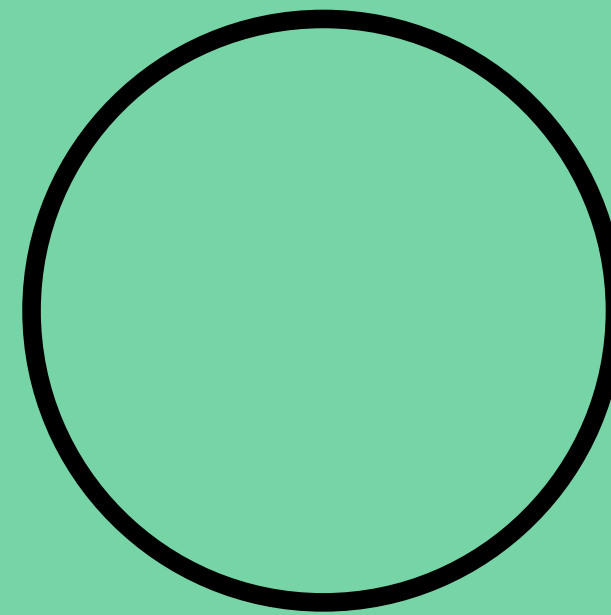
The Iterator interface has the following two key methods: **hasNext()** and **next()**



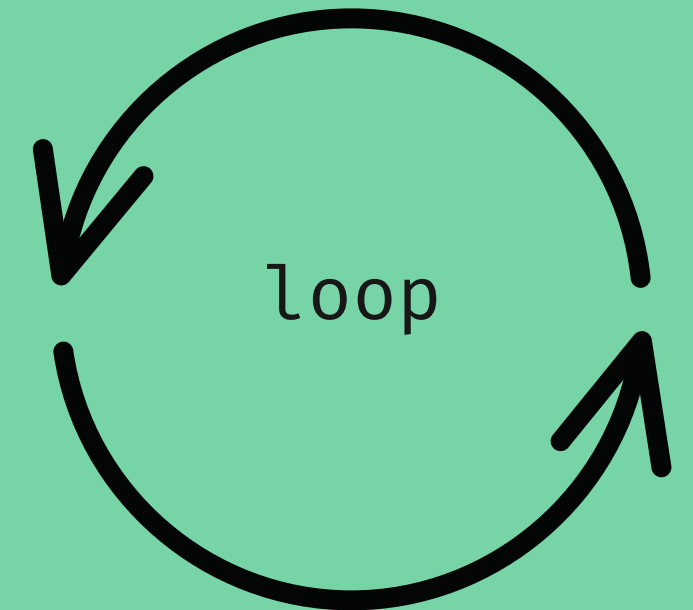
Class Potato

Iterable Interface

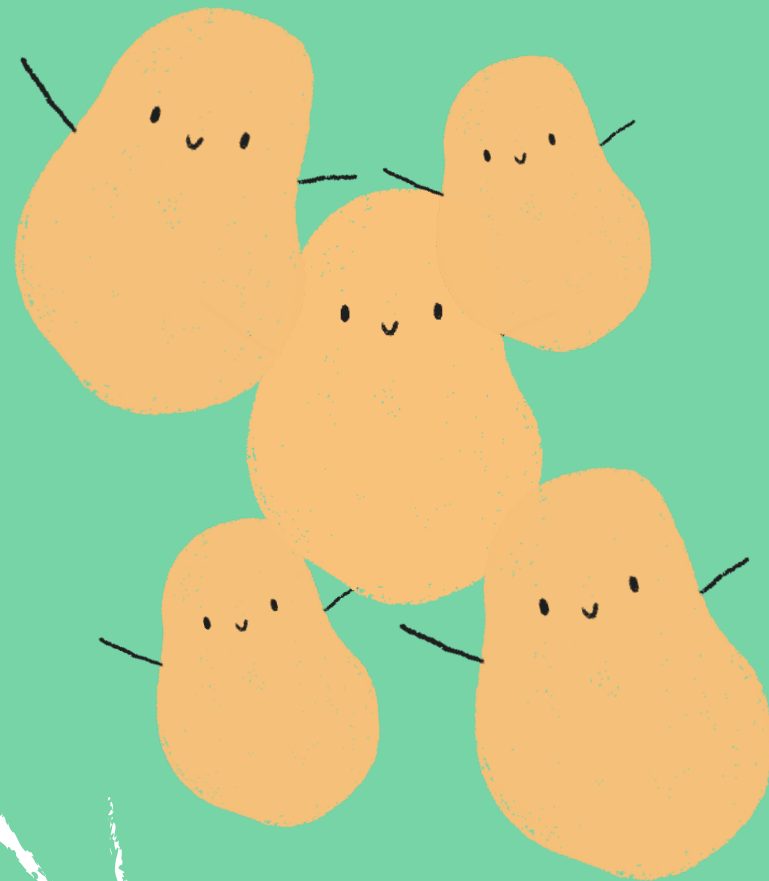
Iterator

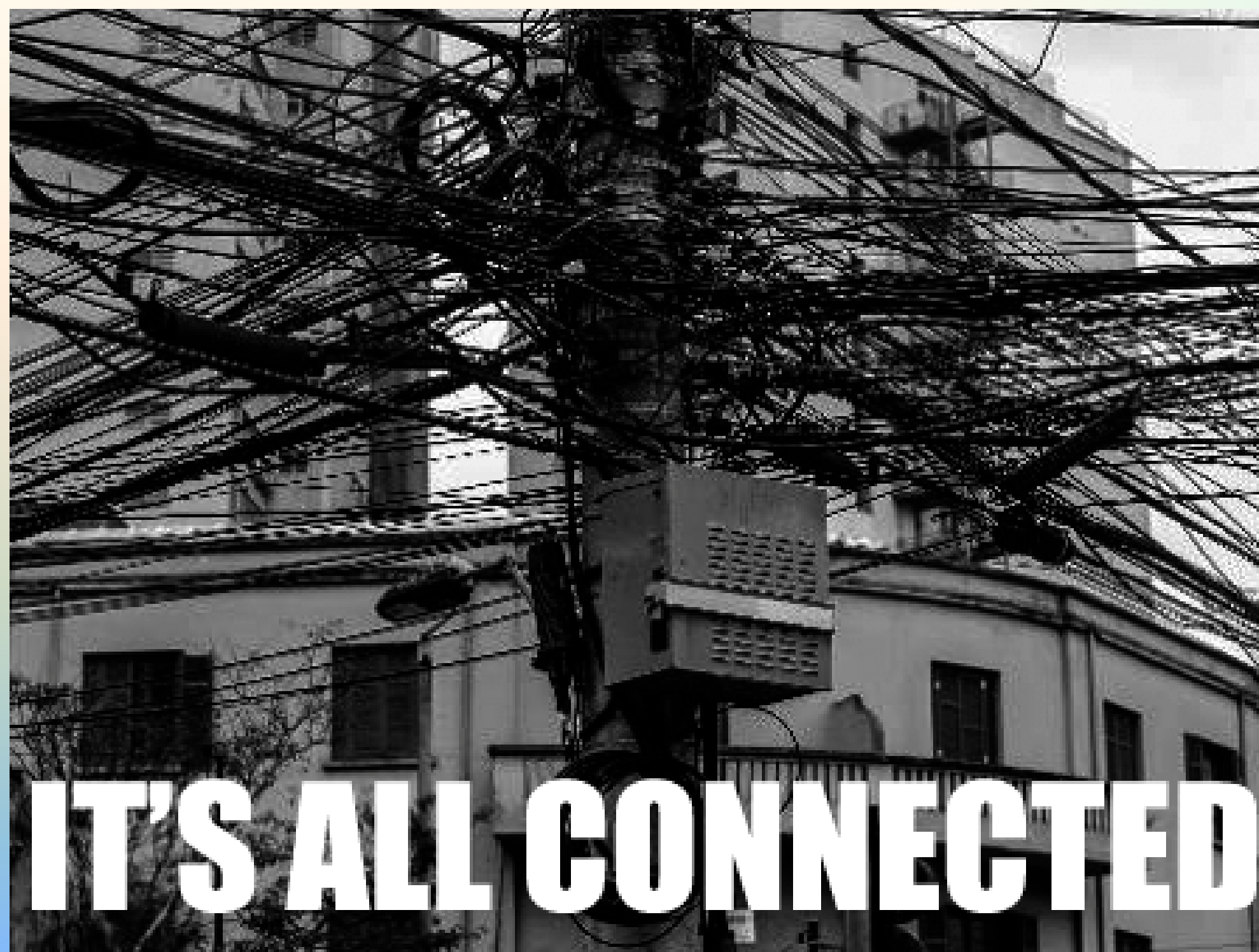


hasNext ();
next ();



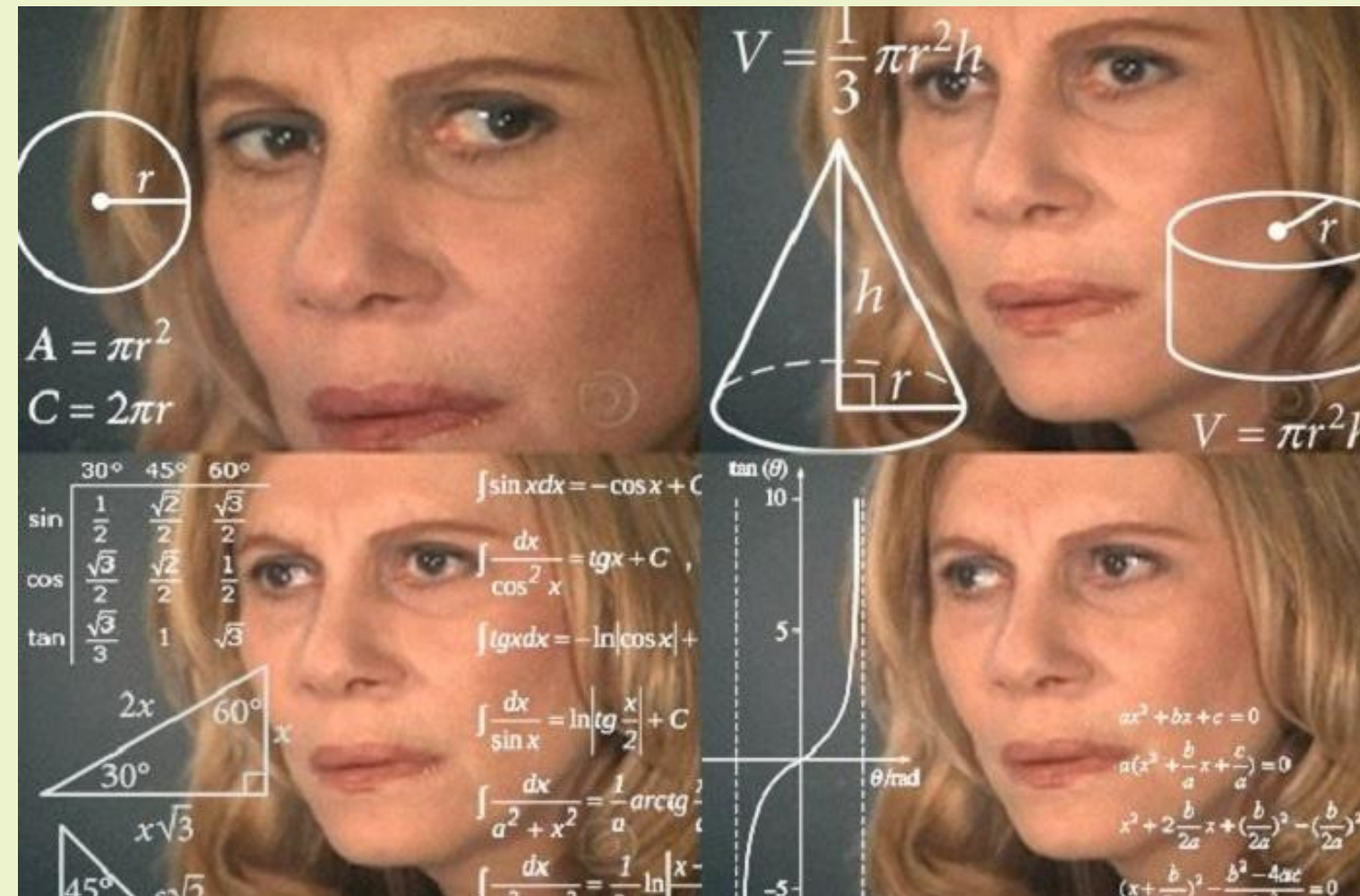
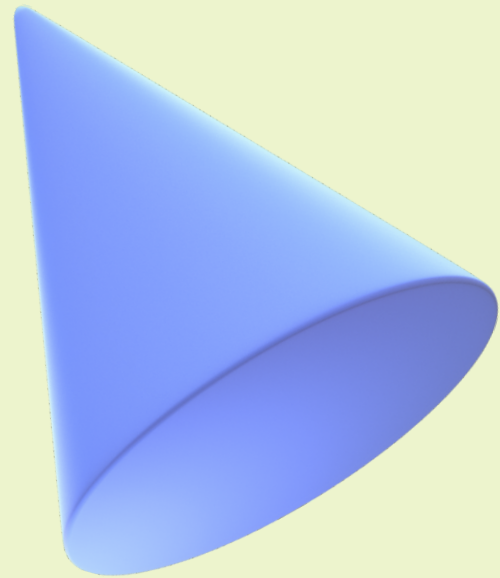
Iterator ()

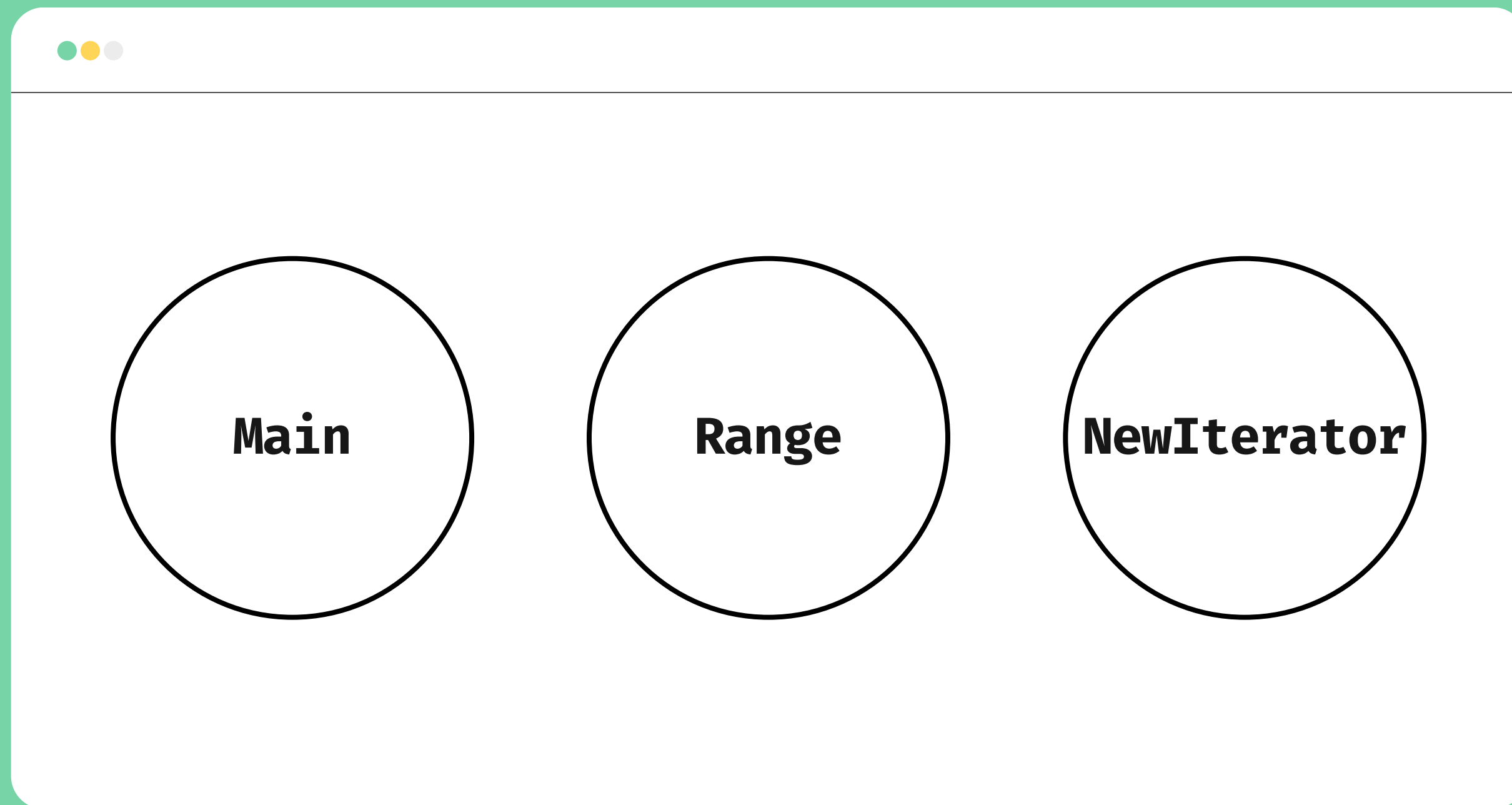




IT'S ALL CONNECTED

Implement an Iterable Integer Range class







```
public class Range implements Iterable<Integer>{

    private int minValue;
    private int maxValue;

    public Range(int minValue, int maxValue) {
        this.minValue = minValue;
        this.maxValue = maxValue;
    }

    @Override
    public Iterator<Integer> iterator() {
        return new NewIterator(minValue, maxValue);
    }
}
```



```
public class NewIterator implements Iterator<Integer> {

    private int minValue;
    private int maxValue;
    private int counter;

    public NewIterator(int minValue, int maxValue) {
        this.minValue = minValue;
        this.maxValue = maxValue;
        this.counter = minValue - 1;
    }

    @Override
    public boolean hasNext() {

        if(counter < maxValue){
            counter++;
            return true;
        }
        return false;
    }

    @Override
    public Integer next() {
        return counter;
    }
}
```



```
public class Main {  
    public static void main(String[] args) {  
  
        Range range = new Range(-1,10);  
        Iterator iterator = range.iterator();  
  
        while (iterator.hasNext())  
  
            System.out.println("The number is: " + iterator.next());  
  
    }  
}
```



```
public class Main {  
    public static void main(String[] args) {  
  
        Range range = new Range(-1,10);  
  
        for (Integer integer : range) {  
            System.out.println("The number is: " + integer);  
        }  
    }  
}
```



```
The number is: -1  
The number is: 0  
The number is: 1  
The number is: 2  
The number is: 3  
The number is: 4  
The number is: 5  
The number is: 6  
The number is: 7  
The number is: 8  
The number is: 9  
The number is: 10
```

```
Process finished with exit code 0
```




**THANK
YOU!**

